Noise-to-Edge Loop

One measured pipeline from damaged training labels to an INT8 model on a CPU. Public MRI data, small 2D models, 30 training runs, every claim checked against a results file.

> **Short answer.** At 40% damaged masks the models lose about 0.5 Dice points. The audit caught 53 of 64 damaged cases, but retraining without them did not help. INT8 left the small model's Dice unchanged and made it 3.7× smaller and 2.2× faster on one CPU thread.

Damage labels
3 error types, Dice 0.8 against the clean mask
0 / 20 / 40%

Train
2 small models, 3 seeds each
30 runs

Audit and filter
existing check drops suspicious cases
53 of 64 caught

Retrain
on the filtered set, same seeds
0.8855 to 0.8927 Dice

INT8 on CPU
post-training quantization, ONNX Runtime
3.7× smaller

Production labels are never perfect, and the model still has to fit on a small device.

Three questions, one pipeline

- How much Dice does a small segmentation model lose when part of its training masks are wrong in known ways?
- How much of that loss does an automatic audit win back by removing suspicious cases?
- What does post-training INT8 cost in Dice, and what does it buy in size and CPU latency?

How I kept it honest

- Predictions P1 to P7 were written in a design file before the main run.
- Split by patient case, never by slice. Test masks are always clean.
- One pilot, one main run, no tuning on the test cases.
- Every number here comes from a results CSV. Failures stay in.

Small on purpose: 2D slices, CPU only, public data.

Data

MSD Task04 Hippocampus, 260 cases, binary mask. 2D axial slices padded to 64×64. Split by case: 160 train, 40 validation, 60 test.

Label damage

Wrong size, shifted mask or extra island, chosen at random per case, each set to Dice 0.8 against the clean mask. 20% or 40% of training cases are damaged. Validation and test masks stay clean.

Models

A: small U-Net from scratch, 4 levels, base 16 channels.
B: U-Net with an ImageNet ResNet18 encoder. A pretrained encoder, not a foundation model.

Audit filter

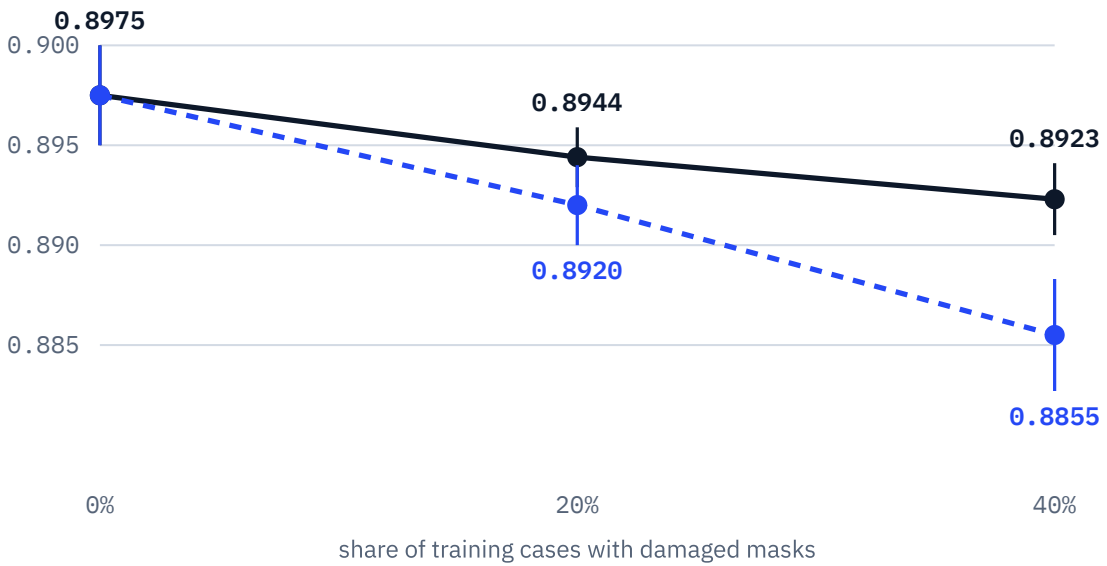
An existing detector scores every training case from its image and mask. The top share equal to the nominal noise rate is dropped, without looking at which cases were really damaged.

Edge step

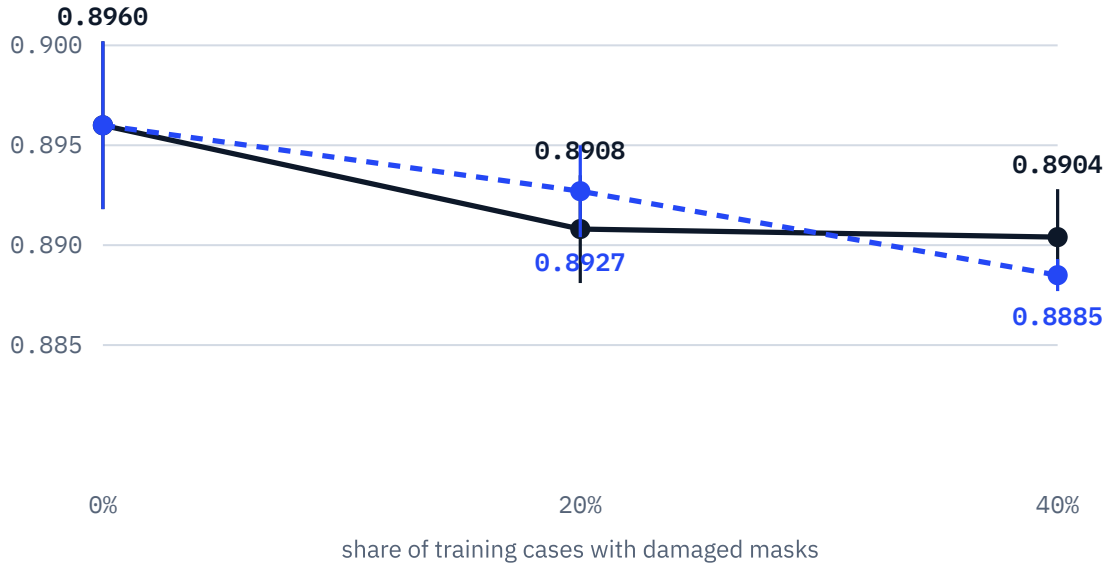
ONNX export, ONNX Runtime static INT8 with 100 training slices, timed on one CPU thread at batch size 1.

With 40% damaged masks, test Dice drops by about half a point.

U-Net from scratch



U-Net with ResNet18 encoder

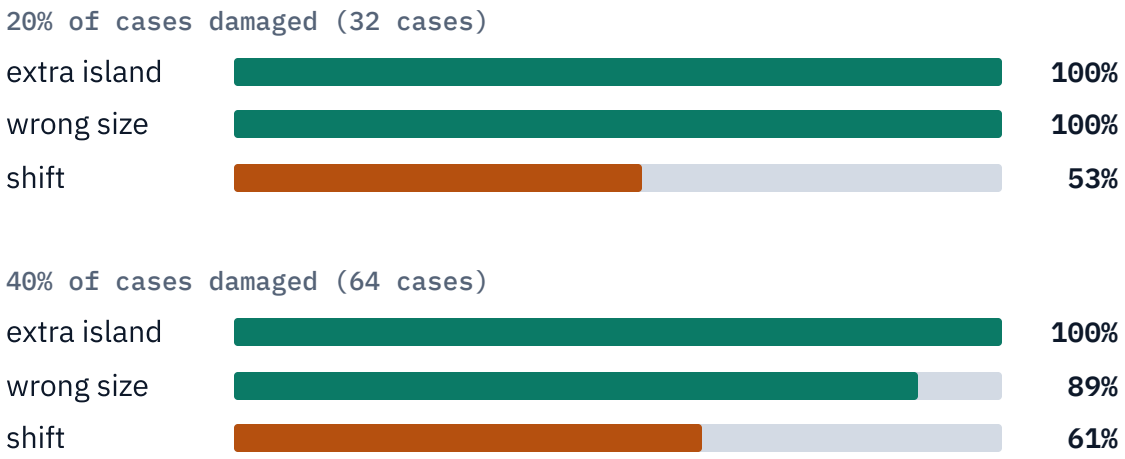


no filter audit-filtered mean ± SD over 3 seeds, clean test cases

Clean to 40% noise: scratch −0.0052, ResNet18 −0.0056 Dice. Seed-to-seed SD is 0.0015 to 0.0042, so the effect is small and only just visible.

The audit finds most damaged cases. Removing them did not bring Dice back.

Share of damaged cases caught, by error type



Overall: 24 of 32 caught at 20%, 53 of 64 at 40%. Shifted masks are the weak spot, as in the earlier audit.

Test Dice after retraining on the filtered set

MODEL	NOISE	CHANGE VS NO FILTER
Scratch	20%	−0.0024
Scratch	40%	−0.0068
ResNet18	20%	+0.0019
ResNet18	40%	−0.0019

Dropping 20% to 40% of the training cases costs about as much as the damaged labels did. At this noise size there was little to recover.

INT8 was free for the small U-Net and cost 0.013 Dice for the larger one.

U-Net from scratch

Test Dice	0.8952 → 0.8952	no change
File size	1.93 → 0.53 MB	3.7× smaller
Latency, median	2.58 → 1.18 ms	2.2× faster



Smallest model, and INT8 left its test Dice unchanged to four decimals.

U-Net with ResNet18 encoder

Test Dice	0.8912 → 0.8779	−0.0133
File size	57.28 → 14.47 MB	4.0× smaller
Latency, median	6.07 → 2.10 ms	2.9× faster



Larger model, larger gain in size and speed, and a visible Dice loss.

Next step before shipping the larger model: check where the 0.0133 Dice goes, for example with a better calibration set or quantization-aware training. Neither was tried here.

Measured on a 4-core cloud CPU, ONNX Runtime 1.29.0, one thread, batch size 1, 1501 test slices. Models: noise 0, seed 0. Post-training quantization, no QAT, not an embedded target.

A toy synthetic generator helped a little, and only when real data followed.

Test Dice on real held-out cases, 3 seeds

(i) 20 real cases only



0.8568 ± 0.0062

(ii) synthetic pretrain, then the same 20 cases



0.8673 ± 0.0059

(iii) synthetic only, no real case



0.3429 ± 0.0094

Why the gap is so large

500 randomised ellipsoid volumes (random pose, size, contrast, blur and noise) are much easier to see than a real hippocampus.

Contrast-to-noise inside versus outside the mask: **2.41** synthetic against **0.55** real.

> Synthetic pretraining plus fine-tuning beat 20 real cases alone by +0.0105 on average, but not in every seed (+0.0145, +0.0173, −0.0003). I report it as a weak positive.

Seven predictions written in advance. Two held, two missed, three held in part.

#	PREDICTION	OUTCOME
P1	40% noise lowers Dice by more than 1 SD	**HELD** drop of 0.0052 (scratch) and 0.0056 (ResNet18, marginal)
P2	Pretrained encoder loses less to noise	**NO** 0.0056 against 0.0052
P3	Filter recovers part of the loss, recall below 0.8	**NO** no recovery, recall 0.83 at 40%
P4	INT8 loses less than 0.01 Dice	**SPLIT** scratch 0.0000 change, ResNet18 −0.0133
P5	INT8 3 to 4× smaller, latency gain unclear	**PARTLY** 3.7× and 4.0× smaller; both faster, I expected no gain for scratch
P6	Synthetic only is clearly worse than real only	**HELD** 0.3429 against 0.8568
P7	Pretrain plus fine-tune beats real only by more than 1 SD	**WEAK** +0.0105 on average, 2 of 3 seeds

Limits: one small public dataset, 2D slices of about 50×35 pixels, no clinical claim. Synthetic damage at Dice 0.8, not matched to real annotator disagreement. One split and one noise draw, so SD covers training seeds only. INT8 numbers come from one model per architecture on a cloud CPU, not an embedded device or a laptop. No QAT, no 3D.